PROJECT MÔN CLOUD COMPUTING

Triển khai ứng dụng quản lý công việc trên cloud

Hình thức: Nhóm 5 sinh viên
Thời gian thực hiện: 10 tuần
Sản phẩm chính: Ứng dụng hoạt động trên Internet, mã nguồn và báo cáo
Nền tảng: AWS, Microsoft Azure, Google Cloud, Oracle Cloud hoặc nền tảng cloud tương đương

---

1. Mục tiêu

Sau khi hoàn thành project, sinh viên có thể:

- Đóng gói một ứng dụng web bằng Docker.
- Triển khai ứng dụng lên một nền tảng cloud.
- Kết nối ứng dụng với cơ sở dữ liệu.
- Quản lý cấu hình và thông tin bí mật bằng biến môi trường.
- Theo dõi log, trạng thái hoạt động và tài nguyên của ứng dụng.
- Giải thích kiến trúc, chi phí và các lựa chọn kỹ thuật của nhóm.

2. Đề bài

Xây dựng và triển khai một **ứng dụng web quản lý công việc cá nhân** trên cloud.

Ứng dụng cần có các chức năng tối thiểu:

1. Tạo tài khoản và đăng nhập.
2. Thêm công việc mới.
3. Xem danh sách công việc.
4. Cập nhật tên, mô tả, thời hạn hoặc trạng thái công việc.
5. Xóa công việc.
6. Lọc công việc theo trạng thái `Chưa hoàn thành` và `Đã hoàn thành`.
7. Lưu dữ liệu trong cơ sở dữ liệu.

Nhóm có thể thay thế đề tài bằng ứng dụng quản lý ghi chú, sinh viên, sách, chi tiêu hoặc sản phẩm. Đề tài thay thế phải có đăng nhập, lưu dữ liệu và các chức năng CRUD tương đương.

3. Phạm vi kỹ thuật bắt buộc

3.1. Ứng dụng

- Có giao diện web và phần xử lý phía server.
- Được phép sử dụng bất kỳ ngôn ngữ hoặc framework nào.
- Có trang hoặc API kiểm tra trạng thái, ví dụ `/health`.
- Có dữ liệu mẫu để giảng viên kiểm thử.

3.2. Docker

- Có `Dockerfile` để đóng gói ứng dụng.
- Image phải build và chạy được trên máy khác.
- Không đưa mật khẩu, access key hoặc file `.env` lên Git.
- Khuyến khích dùng `.dockerignore` và image tag theo phiên bản, ví dụ `v1.0`.

3.3. Cloud

Ứng dụng phải được triển khai bằng ít nhất các thành phần sau:

Thành phần	Yêu cầu
Compute	VM, container service hoặc platform as a service để chạy ứng dụng
Database	MySQL, PostgreSQL, MongoDB hoặc dịch vụ database được quản lý
Observability	Có log ứng dụng và ít nhất một metric hoặc thông tin giám sát

Ứng dụng phải truy cập được từ Internet bằng một URL công khai tại thời điểm chấm. Nếu tắt tài nguyên để tiết kiệm chi phí, nhóm phải có khả năng triển khai lại khi được yêu cầu.

3.4. Bảo mật và chi phí

- Chỉ mở các cổng mạng thật sự cần thiết.
- Mật khẩu và chuỗi kết nối được truyền qua biến môi trường hoặc secret manager.
- Tài khoản cloud bật xác thực đa yếu tố nếu nền tảng hỗ trợ.
- Có cảnh báo ngân sách hoặc ảnh chụp trang theo dõi chi phí.
- Không sử dụng tài khoản quản trị cao nhất cho hoạt động hằng ngày nếu có thể tạo tài khoản/quyền riêng.

4. Hướng dẫn thực hiện

Bước 1 — Chọn công nghệ và phân công

Chọn một bộ công nghệ mà nhóm đã quen thuộc. Ví dụ:

- Frontend: HTML/CSS/JavaScript, React hoặc Vue.
- Backend: Node.js/Express, Python/Flask/FastAPI, Java/Spring Boot hoặc .NET.
- Database: PostgreSQL, MySQL hoặc MongoDB.
- Cloud: một nền tảng mà nhóm có tài khoản và credit phù hợp.

Phân công tối thiểu các vai trò: phát triển ứng dụng, database, cloud/DevOps và kiểm thử/tài liệu. Mọi thành viên vẫn phải hiểu kiến trúc chung.

Bước 2 — Hoàn thiện ứng dụng ở máy cá nhân

1. Thiết kế bảng dữ liệu hoặc collection.
2. Cài đặt đăng nhập và các chức năng CRUD.
3. Kiểm thử các luồng chính và trường hợp dữ liệu không hợp lệ.
4. Thêm endpoint `/health` trả về trạng thái của ứng dụng.
5. Ghi cấu hình mẫu trong `.env.example`, không ghi giá trị bí mật thật.

Kết quả cần đạt: ứng dụng chạy ổn định trên máy của ít nhất hai thành viên.

Bước 3 — Đóng gói bằng Docker

1. Viết `Dockerfile`.
2. Build image với tag phiên bản.
3. Chạy container và truyền cấu hình bằng biến môi trường.
4. Kiểm tra giao diện, kết nối database và endpoint `/health`.
5. Đẩy image lên container registry nếu phương án triển khai yêu cầu.

Ví dụ quy trình tổng quát:

```
docker build -t ten-nhom/task-app:v1.0 .
docker run --env-file .env -p 8080:8080 ten-nhom/task-app:v1.0
```

Nhóm phải điều chỉnh cổng, tên image và biến môi trường theo ứng dụng của mình.

Bước 4 — Tạo tài nguyên cloud

1. Chọn region gần người dùng và ghi rõ lý do lựa chọn.
2. Tạo compute service để chạy ứng dụng.
3. Tạo database hoặc cấu hình database phù hợp.
4. Thiết lập firewall/security group chỉ cho phép các kết nối cần thiết.
5. Cấu hình biến môi trường hoặc secret trên dịch vụ cloud.
6. Triển khai image và kiểm tra log khởi động.

Không chụp hoặc đưa access key, mật khẩu và connection string đầy đủ vào báo cáo.

Bước 5 — Kiểm thử sau triển khai

Nhóm thực hiện và ghi lại kết quả của các tình huống sau:

Mã	Tình huống	Kết quả mong đợi
T1	Mở URL công khai	Trang web hiển thị bình thường
T2	Đăng ký và đăng nhập	Người dùng truy cập được chức năng cá nhân
T3	Thêm, sửa, xóa công việc	Dữ liệu thay đổi đúng trong database
T4	Khởi động lại ứng dụng	Dữ liệu vẫn còn
T5	Gọi `/health`	Trả về trạng thái thành công
T6	Gửi dữ liệu không hợp lệ	Ứng dụng báo lỗi phù hợp, không bị dừng
T7	Xem log	Tìm được request hoặc lỗi thử nghiệm

Bước 6 — Thiết lập giám sát và chi phí

- Hiển thị được log của ứng dụng trên nền tảng cloud.
- Theo dõi ít nhất một thông tin như CPU, bộ nhớ, số request hoặc lỗi HTTP.
- Tạo budget alert hoặc cơ chế theo dõi chi phí phù hợp.
- Ước tính chi phí vận hành trong một tháng và ghi rõ các giả định.

Bước 7 — Hoàn thiện tài liệu và dọn tài nguyên

- Kiểm tra một thành viên khác có thể triển khai theo `README.md`.
- Gắn tag hoặc release cho phiên bản được chấm.
- Sao lưu các ảnh minh chứng cần thiết.
- Sau khi chấm, xóa compute, database, IP tĩnh, disk và các tài nguyên tính phí không còn sử dụng.

6. Cấu trúc repository gợi ý

```
project-cloud/
├── frontend/                # Nếu frontend tách riêng
├── backend/                 # Mã nguồn backend
├── docs/
│   ├── architecture.png     # Sơ đồ kiến trúc
│   └── test-results.md      # Kết quả kiểm thử
├── .env.example             # Chỉ chứa tên biến và giá trị mẫu
```

```
├── .gitignore
├── Dockerfile
├── docker-compose.yml        # Không bắt buộc
└── README.md
```

`README.md` cần có:

- Mô tả project và thành viên nhóm.
- Sơ đồ kiến trúc.
- Công nghệ và dịch vụ cloud đã sử dụng.
- Cách chạy trên máy cá nhân.
- Cách build Docker image.
- Các bước triển khai lên cloud.
- Các biến môi trường cần cấu hình.
- Cách kiểm thử và xem log.
- Cách xóa tài nguyên cloud.

7. Sản phẩm cần nộp

Mỗi nhóm nộp một thư mục hoặc biểu mẫu chứa:

1. Link repository Git.
2. Link ứng dụng đang hoạt động.
3. Báo cáo PDF từ 10–20 trang.
4. Slide thuyết trình.
5. Video demo từ 5–7 phút.
6. Bảng phân công và tỷ lệ đóng góp của từng thành viên.

Báo cáo cần trình bày:

- Bài toán và chức năng của ứng dụng.
- Sơ đồ kiến trúc và luồng dữ liệu.
- Các dịch vụ cloud đã sử dụng và lý do lựa chọn.
- Quy trình Docker hóa và triển khai.
- Cách quản lý cấu hình, secret và quyền truy cập.
- Bảng kết quả kiểm thử kèm minh chứng.
- Log hoặc dashboard giám sát.
- Bảng ước tính chi phí trong một tháng.
- Khó khăn, cách xử lý và bài học kinh nghiệm.

Điểm cộng

Tổng điểm sau khi cộng không vượt quá 10:

- Có pipeline CI/CD tự động: **+0,5 điểm**.
- Có HTTPS và tên miền: **+0,25 điểm**.
- Có sao lưu và khôi phục database được kiểm chứng: **+0,25 điểm**.
- Có Infrastructure as Code bằng Terraform hoặc công cụ tương đương: **+0,5 điểm**.
- Có kiểm thử tải và phân tích kết quả: **+0,25 điểm**.

8. Quy định

- Không chấp nhận ứng dụng chỉ chạy trên máy cá nhân.
- Không đưa mật khẩu, access key, private key hoặc file `.env` lên repository.
- Được tham khảo mã nguồn bên ngoài nhưng phải ghi rõ nguồn và giải thích được phần đã sử dụng.
- Không bắt buộc tự viết toàn bộ giao diện; trọng tâm là triển khai và vận hành trên cloud.
- Mỗi thành viên phải tham gia trình bày và trả lời câu hỏi.
- Điểm cá nhân có thể khác điểm nhóm dựa trên đóng góp và phần vấn đáp.
- Nhóm chịu trách nhiệm theo dõi chi phí phát sinh và dọn tài nguyên sau khi chấm.

12. Checklist trước khi nộp

- [ ] URL công khai truy cập được.
- [ ] Tài khoản demo và dữ liệu mẫu hoạt động.
- [ ] Repository không chứa secret.
- [ ] Docker image build và chạy thành công.
- [ ] Database lưu dữ liệu bền vững.
- [ ] `/health` trả về trạng thái hợp lệ.
- [ ] Có log và ít nhất một thông tin monitoring.
- [ ] Có sơ đồ kiến trúc đúng với hệ thống thực tế.
- [ ] `README.md` đủ để một người khác triển khai lại.
- [ ] Báo cáo có kết quả kiểm thử và ước tính chi phí.
- [ ] Đã chuẩn bị phương án dọn toàn bộ tài nguyên sau khi chấm.